



Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА № 5
з дисципліни «Технології розроблення програмного забезпечення»
Тема: «Патерни проектування»

Виконала:

Студентка групи ІА-31

Соколова Поліна

Мета: Вивчити структуру шаблонів «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та навчитися застосовувати їх в реалізації програмної системи.

15. Е-mail клієнт

Е-mail клієнт - це програма для управління електронною поштою, яка повинна забезпечувати:

- Підтримку протоколів POP3, SMTP, IMAP
- Автоналаштування для українських провайдерів (Gmail, ukr.net, i.ua)
- Організацію повідомлень у папки/категорії/важливість
- Роботу з чернетками
- Управління прикріпленнями до повідомлень

Патерни проектування для виконання:

singleton, builder, decorator, template method, interpreter, client-server.

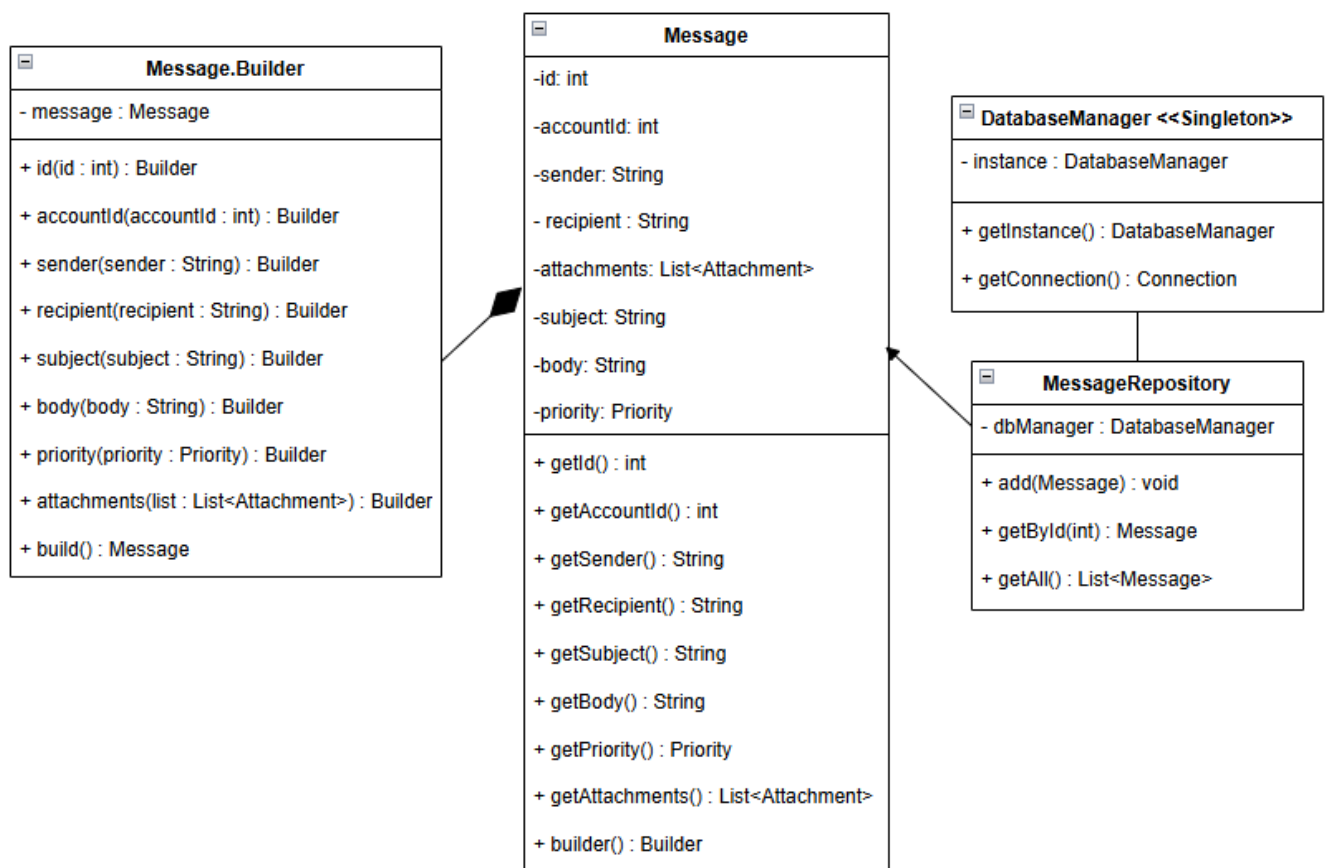


Рис.1 Діаграма класів, яка представляє використання шаблону «Builder»

```

private Message(Builder builder) { 1 usage
    this.id = builder.id;
    this.accountId = builder.accountId;
    this.sender = builder.sender;
    this.recipient = builder.recipient;
    this.subject = builder.subject;
    this.body = builder.body;
    this.priority = builder.priority;
    this.attachments = builder.attachments;
}

public static Builder builder() { 3 usages
    return new Builder();
}

public static class Builder { 11 usages
    private int id; 2 usages
    private int accountId; 2 usages
    private String sender; 2 usages
    private String recipient; 2 usages
    private String subject; 2 usages
    private String body; 2 usages
    private Priority priority; 2 usages
    private List<Attachment> attachments; 2 usages

    public Builder id(int id) { no usages
        this.id = id;
        return this;
    }

    public Builder accountId(int accountId) { no usages
        this.accountId = accountId;
        return this;
    }

    public Builder sender(String sender) { no usages
        this.sender = sender;
        return this;
    }

    public Builder recipient(String recipient) { no usages
        this.recipient = recipient;
        return this;
    }

    public Message build() { 3 usages
        return new Message( builder: this);
    }
}

```

1

```

Message msg = Message.builder()
    .id(rs.getInt( columnLabel: "id"))
    .accountId(rs.getInt( columnLabel: "accountId"))
    .sender(rs.getString( columnLabel: "sender"))
    .recipient(rs.getString( columnLabel: "recipient"))
    .subject(rs.getString( columnLabel: "subject"))
    .body(rs.getString( columnLabel: "body"))
    .priority(rs.getString( columnLabel: "priority") != null
        ? Priority.valueOf(rs.getString( columnLabel: "priority"))
        : null)
    .build();

```

2

Рис.2 Фрагменти коду по реалізації шаблону
(1– клас Messenger, 2 – Messenger Repository)

Висновок: у даній лабораторній роботі було реалізовано шаблон «Builder» для створення об'єктів моделі Message, що є частиною системи E-mail клієнта. Шаблон відокремлює процес конструювання об'єкта від його фінального представлення, дозволяючи створювати різні представлення одного об'єкта за допомогою одного й того ж процесу побудови. Після застосування Builder поля класу Message стали незмінними (final), що гарантує їх встановлення лише один раз при створенні об'єкта. Публічні сеттери було видалено, залишивши лише геттери для читання даних. Створення об'єктів тепер виконується поетапно через внутрішній клас Builder, який акумулює значення параметрів та створює новий екземпляр Message при виклику методу build().

Відповіді на питання:

1. Яке призначення шаблону «Адаптер»?

Шаблон «Адаптер» використовується для адаптації інтерфейсу одного об'єкта до іншого. Він дозволяє об'єктам з несумісними інтерфейсами працювати разом, створюючи уніфікований інтерфейс для роботи з різними реалізаціями.

3. Які класи входять в шаблон «Адаптер», та яка між ними взаємодія?

Target (інтерфейс) — визначає уніфікований інтерфейс, який використовує клієнт

Client — працює з об'єктами через інтерфейс Target

Adaptee — існуючий клас з несумісним інтерфейсом

Adapter — адаптує інтерфейс Adaptee до Target, містить посилання на Adaptee і реалізує методи Target, викликаючи методи Adaptee.

4. Яка різниця між реалізацією «Адаптера» на рівні об'єктів та на рівні класів?

На рівні об'єктів (композиція): Adapter містить посилання на об'єкт Adaptee і делегує йому виклики. На рівні класів (множинне успадкування): Adapter успадковується одночасно від Target і Adaptee (можливо не у всіх мовах програмування, наприклад, Java не підтримує множинне успадкування класів).

5. Яке призначення шаблону «Будівельник»?

Шаблон «Будівельник» використовується для відділення процесу створення об'єкта від його представлення. Він дозволяє створювати складні об'єкти покроково та отримувати різні представлення об'єкта за допомогою одного процесу побудови.

7. Які класи входять в шаблон «Будівельник», та яка між ними взаємодія?

Builder (інтерфейс) — визначає абстрактні методи для створення частин об'єкта

ConcreteBuilder — реалізує методи Builder для створення конкретного типу продукту

Director — керує процесом побудови, викликаючи методи Builder у потрібній послідовності

Product — складний об'єкт, який створюється

8. У яких випадках варто застосовувати шаблон «Будівельник»?

- об'єкт має складний процес створення з багатьма кроками
- потрібно створювати різні представлення одного об'єкта
- багато параметрів конструктора (5+), особливо опціональних
- потрібна валідація або спеціальна логіка при створенні об'єкта
- потрібно відокремити логіку конструювання від бізнес-логіки

9. Яке призначення шаблону «Команда»?

Шаблон «Команда» перетворює звичайний виклик методу в окремий клас-об'єкт. Це дозволяє параметризувати об'єкти операціями, ставити операції в чергу, логувати їх виконання, підтримувати скасування операцій.

11. Які класи входять в шаблон «Команда», та яка між ними взаємодія?

Command (інтерфейс) — визначає метод execute() для виконання команди

ConcreteCommand — реалізує execute(), містить посилання на Receiver і викликає його методи

Receiver — виконує фактичну роботу (бізнес-логіка)

Invoker — ініціює виконання команди, не знає деталей її реалізації

Client — створює команди та налаштовує їх

12. Розкажіть як працює шаблон «Команда».

- 1) Client створює об'єкт ConcreteCommand та передає йому Receiver
- 2) Invoker зберігає посилання на Command
- 3) Коли потрібно виконати дію, Invoker викликає `command.execute()`
- 4) ConcreteCommand перенаправляє виклик методу `receiver.action()`
- 5) Receiver виконує фактичну роботу

Приклад: користувач натискає кнопку (Invoker) → викликається команда "Зберегти" (ConcreteCommand) → команда викликає метод `save()` у об'єкта Document (Receiver).

13. Яке призначення шаблону «Прототип»?

Шаблон «Прототип» використовується для створення нових об'єктів шляхом копіювання (клонування) існуючого об'єкта-прототипу. Це дозволяє уникнути складного процесу створення об'єктів та зменшує залежність від конкретних класів.

15. Які класи входять в шаблон «Прототип», та яка між ними взаємодія?

Prototype (інтерфейс) — визначає метод `clone()` для клонування себе

ConcretePrototype — реалізує метод `clone()`, створюючи копію самого себе

Client — створює нові об'єкти шляхом виклику `clone()` на прототипі

Взаємодія: Client не створює об'єкти через конструктори, а клонує існуючі прототипи. Це дозволяє динамічно додавати нові типи об'єктів без зміни коду Client.

Шаблон «Ланцюжок відповідальності»

16. Які можна привести приклади використання шаблону «Ланцюжок відповідальності»?

Формування контекстного меню — кожен візуальний компонент додає свої пункти меню та передає запит батьківському елементу

Система логування — запис логів різних рівнів (DEBUG → INFO → WARNING → ERROR), кожен обробник вирішує чи обробляти повідомлення

Фільтри HTTP-запитів — автентифікація → авторизація → валідація → обробка запиту

Обробка помилок — кожен обробник перевіряє чи може обробити помилку, якщо ні — передає далі

Система підписання документів — співробітник → менеджер → начальник відділу → директор

Обробка подій UI — подія спочатку обробляється найглибшим елементом, потім піднімається вгору по ієрархії (event bubbling)

Валідація даних форми — перевірка формату → перевірка на порожні поля → перевірка бізнес-правил.